

LoRa Studio User Guide

Welcome to LoRa® Studio v1.7.4!

LoRa Studio is an application that demonstrates the capabilities and features of Semtech® radio chips. It allows users to configure, test, and visualize the performances of the Gen 4 series in various scenarios. All this without writing even one line of code!

Please make sure you are up to date with the latest version by visiting the [LoRa PLUS webpage!](#)

Getting Started

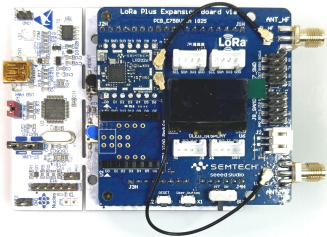
Installing and updating LoRa Studio

On Windows, running the installer to update an existing installation now shows a **"Previous Versions"** page listing every LoRa Studio version currently installed on your machine, each with its own checkbox, along with **"Select All"** / **"Deselect All"** buttons. Any version you check is removed once the new installation completes, making it easy to clean up old versions you no longer need.

Supported development kits

LoRa Studio supports different *types* of development kits, one based on STM32 MCU and the two others based on Nordic nRF54. As you will see in the following sections, the kit name is composed of the *part number* of the radio chip followed by the development kit *type*.

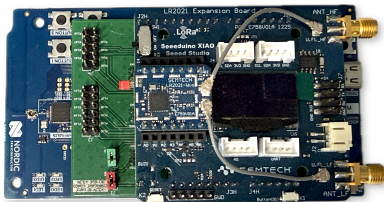
- [LoRa Plus™ ARDUINO® LEGACY \(STM32®\)](#)



Based on the STM32L4 platform, this dev kit is composed of a Nucleo® STM32L476RG, an adaptation board to handle the Arduino pinning, and the radio expansion board. This kit comes as 3 different part numbers for each of the radio chip, depending on the region:

- [LR2021EVK1XBS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2021, 868MHz for Europe
- [LR2021EVK1XCS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2021, 915MHz for North America
- [LR2021EVK1XGS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2021, 490MHz for China and Asia
- [LR2022EVK1XBS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2022, 868MHz for Europe
- [LR2022EVK1XCS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2022, 915MHz for North America
- [LR2022EVK1XGS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2022, 490MHz for China and Asia
- [LR2012EVK1XBS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2012, 868MHz for Europe
- [LR2012EVK1XCS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2012, 915MHz for North America
- [LR2012EVK1XGS1_ARDUINO_LEGACY](#) : LoRa Plus™ Development Kit, LR2012, 490MHz for China and Asia

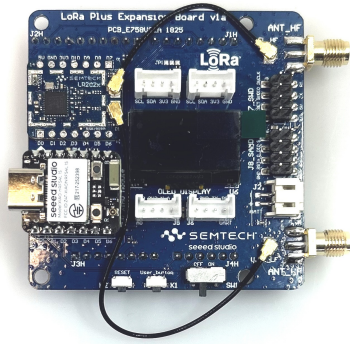
- [LoRa Plus™ ARDUINO nRF54L15 Dev Kit](#)



Based on the nRF54L15 platform, this dev kit is composed of a nRF54L15-DK, an adaptation board to handle the Arduino pinning and the radio expansion board: This kit comes as 3 different part numbers for each of the radio chip, depending on the region:

- [LR2021EVK1XBS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2021, 868MHz for Europe
- [LR2021EVK1XCS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2021, 915MHz for North America
- [LR2021EVK1XGS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2021, 490MHz for China and Asia
- [LR2022EVK1XBS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2022, 868MHz for Europe
- [LR2022EVK1XCS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2022, 915MHz for North America
- [LR2022EVK1XGS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2022, 490MHz for China and Asia
- [LR2012EVK1XBS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2012, 868MHz for Europe
- [LR2012EVK1XCS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2012, 915MHz for North America
- [LR2012EVK1XGS1_ARDUINO](#) : LoRa Plus™ Development Kit, LR2012, 490MHz for China and Asia

- [LoRa Plus™ XIAO Dev Kit](#)



Based on the Seeeduino XIAO platform, this dev kit is composed of the latest integration of the nRF54L15 MCU in the compact XIAO form factor, and the LR2021 radio, either as a module or as an expansion board. This kit comes as 3 different part numbers for each radio chip, depending on the region:

- LR2021EVK1XBS1_XIAO : LoRa Plus™ Development Kit, LR2021, 868MHz for Europe
- LR2021EVK1XCS1_XIAO : LoRa Plus™ Development Kit, LR2021, 915MHz for North America
- LR2021EVK1XGS1_XIAO : LoRa Plus™ Development Kit, LR2021, 490MHz for China and Asia
- LR2022EVK1XBS1_XIAO : LoRa Plus™ Development Kit, LR2022, 868MHz for Europe
- LR2022EVK1XCS1_XIAO : LoRa Plus™ Development Kit, LR2022, 915MHz for North America
- LR2022EVK1XGS1_XIAO : LoRa Plus™ Development Kit, LR2022, 490MHz for China and Asia
- LR2012EVK1XBS1_XIAO : LoRa Plus™ Development Kit, LR2012, 868MHz for Europe
- LR2012EVK1XCS1_XIAO : LoRa Plus™ Development Kit, LR2012, 915MHz for North America
- LR2012EVK1XGS1_XIAO : LoRa Plus™ Development Kit, LR2012, 490MHz for China and Asia

Drivers Installation

• STM32L4 based kits

On Windows, to make sure that you can properly connect to this kit, you need to install the ST-LINK USB driver available here: [ST-LINK USB driver](#).

On Linux, you can install the *stlink* tool suite following the instruction on [the GitHub page](#) but make sure that your OS version is [supported](#).

Otherwise, you need to create a udev rule to allow non-root access to the device:

Create a file named `/etc/udev/rules.d/49-stlinkv2-1.rules` with the following content:

```
SUBSYSTEMS=="usb", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374a", \
    MODE=="0666", \
    SYMLINK+="stlinkv2-1"
SUBSYSTEMS=="usb", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="374b", \
    MODE=="0666", \
    SYMLINK+="stlinkv2-1"
SUBSYSTEMS=="usb", ATTRS{idVendor}=="0483", ATTRS{idProduct}=="3752", \
    MODE=="0666", \
    SYMLINK+="stlinkv2-1"
```

Copy

Then reload the udev rules with the command:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Copy

After this, unplug and replug the device. You should now have access to it without needing root privileges.

In both cases, once properly installed, you should see a mass storage device usually named NUCLEO_L476RG when plugging the kit to your computer. You should also see a serial port associated with the device.

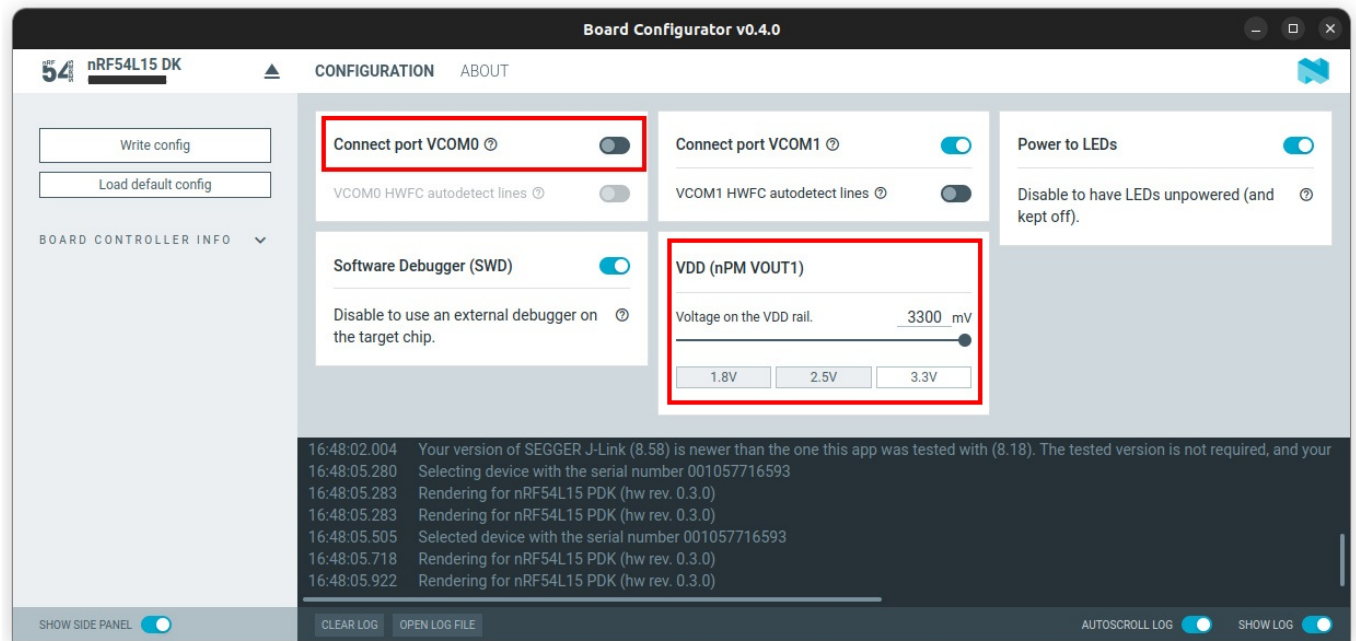
• nRF54L15-DK based kits

To properly handle the Nordic Semiconductor® dev kit, you need to install the [nRF Connect for Desktop tool](#) from Nordic website.

Since the nRF54L15-DK is based on a J-Link debugger, you also need to install the [J-Link driver](#) from Segger website.

Both software packages are available for both Windows and Linux.

Once both software packages are installed, plug the nRF54-DK to your computer. You should see 2 serial ports associated with the device. You will need to deactivate the first one (the one with the lowest number) and set the power voltage to 3.3V as shown below:



Make sure to write back the change by clicking on the *Write Config* button. You can now close the nRF Connect application for Desktop.

You should now be able to use LoRa Studio with the nRF54L15-DK based kits.

- [Seeeduino XIAO based kits](#)

On Windows, good news : nothing to do! The module is plug and play.

On Linux, you need to create a udev rule to allow non-root access to the device:

Create a file named `/etc/udev/rules.d/99-xiao.rules` with the following content:

```
SUBSYSTEMS=="usb", ATTRS{idVendor}=="2886", ATTRS{idProduct}=="0066", \
    MODE:="0666", \
    SYMLINK+="nrf54"
ATTRS{idVendor}=="2886", ATTRS{idProduct}=="[08]02d", MODE="0666", ENV{ID_MM_DEVICE_IGNORE}="1", \
    ENV{ID_MM_PORT_IGNORE}="1"
```

Copy

Then reload the udev rules with the command:

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

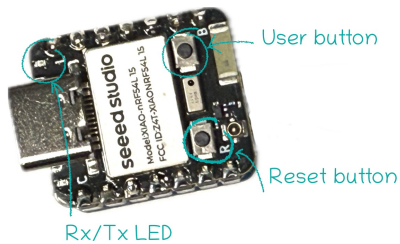
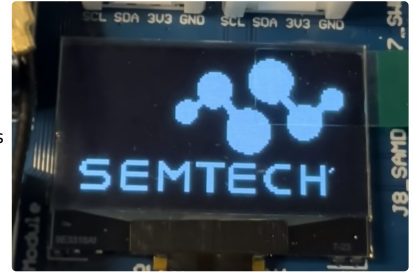
Copy

After this, unplug and replug the device. You should now have access to it without needing root privileges.

Using LoRa Studio

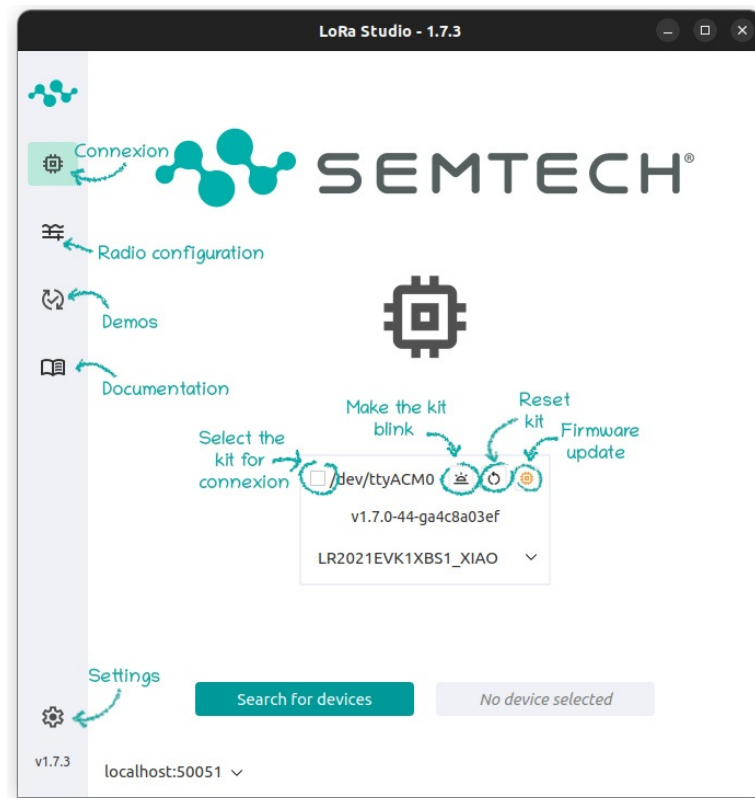
Connecting your development kit

Once the drivers properly installed, you can connect your kits to your computer. If the kit is equipped with the small OLED screen, you shall see a Semtech logo indicating that the kit is powered on. If it does not appear, please reset manually your kit by pressing the reset button.



Note: If you have a Seeedduino XIAO based kit, you will have to reset with the button on the XIAO board, not the one on the expansion board.

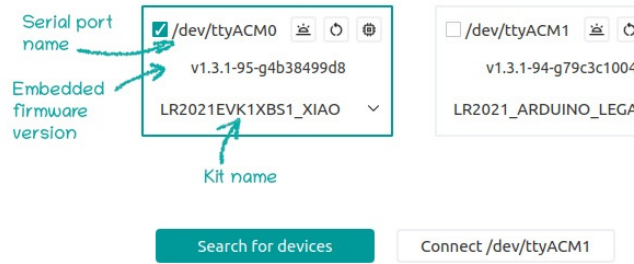
Launch LoRa Studio, here is what it looks like:



At launch, the three first times you will be redirected to this documentation page. After that you will see the [Connexion Page](#). Until you connect a kit, you can only navigate to the documentation page or stay on the connexion page.

If you have multiple kits connected you can identify which serial port is associated with which kit by clicking on the  button. This will make LEDs blink on the corresponding kit.

You can also reset your kit using the  button.

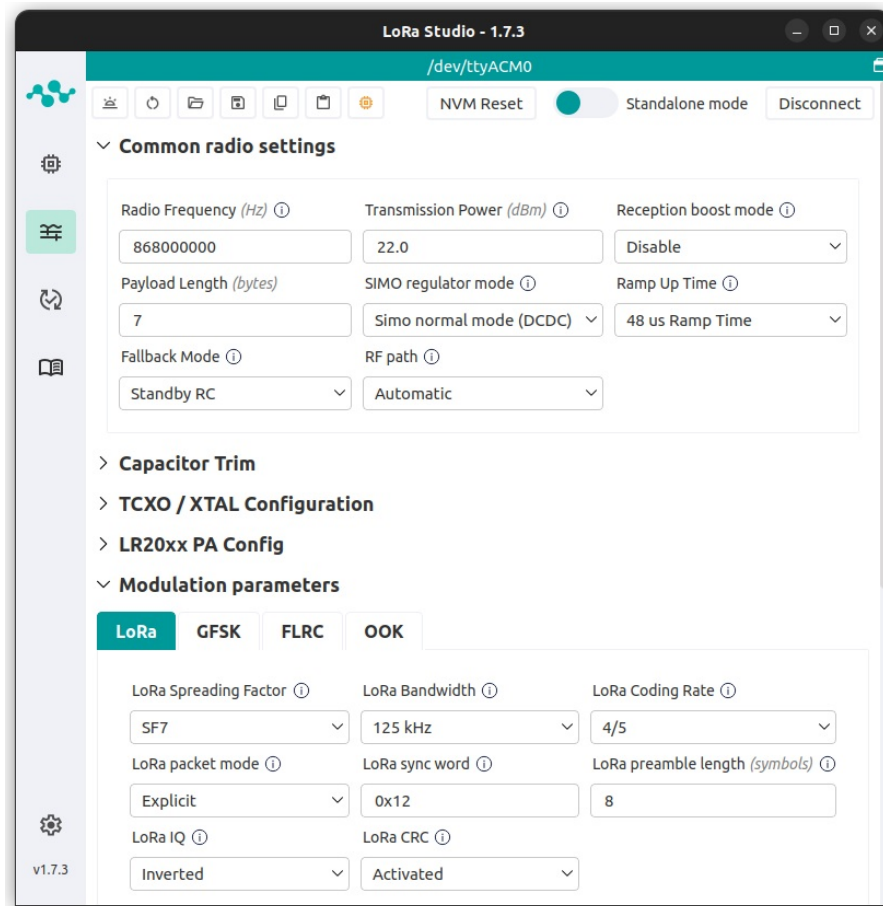


At startup, LoRa Studio detects the connected development kit(s) and displays the kit info as shown above. If no device is detected, make sure that the kit is properly connected and that the drivers are correctly installed or see the [troubleshooting](#) section below.

Important Note about Kit Name: The Kit type (meaning the MCU it is based on) is automatically detected **BUT** the part number must be selected manually. If the exact reference is not correctly set, LoRa Studio may not be able to operate the kit's radio properly leading to unexpected behavior. Please verify that the kit name matches the development kit you are using before proceeding with any operations. Please refer to the [Supported Development Kits](#) section above for the correct kit names.

Once connected, LoRa Studio switches automatically to the [Radio Configuration](#) page.

Configure the radio



From this page, you can configure the radio parameters of your kit. The page is organized into collapsible sections; which ones are shown, and the exact fields inside them, depends on the radio family of your shield (see [Annex A](#) for the full field reference).

Some parameter fields display an info icon; hovering over it shows a preview of the relevant section of the radio datasheet, with a link to open the full PDF at that page. LoRa Studio can also automatically check for a newer revision of the datasheet and prompt you to download it (see the [Settings](#) page to control this check).

Common radio settings

The main parameters shared by every modulation: **Radio Frequency**, **Transmission Power**, **Reception boost mode**, **Payload Length**, **SIMO regulator mode**, **Ramp Up Time**, **Fallback Mode**, and **RF path** (a dropdown to force the RF path used by the radio — *Automatic*, *Low Frequency*, or *High Frequency*). Input fields here are validated as you type, and some automatically adjust to keep your configuration consistent — for example, the maximum accepted *Payload Length* depends on the modulation currently selected below.

Capacitor Trim

Advanced, optional fine-tuning of the crystal oscillator's load capacitance (**XTA** / **XTB** trim values) and an extra XTAL startup delay. Leave this section disabled unless you are calibrating a specific board against measurements taken from the datasheet's procedure.

▼ Capacitor Trim

☐ Enable Capacitor Trim

XTA Trim ⓘ
0x00

XTB Trim ⓘ
0x00

Additionalnal startup Time (μs) ⓘ
0

TCXO / XTAL Configuration

Lets you override the shield's default oscillator. Enable it with the "*Enable TCXO / XTAL Configuration*" checkbox, then choose the *Oscillator Type* (TCXO or XTAL); when TCXO is selected, the *TCXO Supply Voltage* dropdown becomes active to match the voltage your shield's TCXO expects.

▼ TCXO / XTAL Configuration

☐ Enable TCXO / XTAL Configuration

Oscillator Type ⓘ
XTAL ▼

TCXO Supply Voltage ⓘ
3.0 V ▼

LR20xx PA Config (LR11xx PA Config on LR11xx shields)

Raw power-amplifier register control, as an alternative to letting **Transmission Power** drive the PA automatically. Enable it to set the PA path (Low/High Frequency) and the PA's raw power, duty-cycle and slice registers directly, using the values documented in your radio's datasheet. Intended for advanced use cases needing precise PA characterization; most users can leave it disabled.

▼ LR20xx PA Config

☐ Enable Custom Power Configuration

TX power ⓘ

0

PA Duty Cycle LF ⓘ

6

PA Slices LF ⓘ

7

PA Duty Cycle HF ⓘ

16

PA Slices HF ⓘ

LF ▼

Modulation parameters

A set of tabs — **LoRa**, **GFSK**, **FLRC**, **OOK** — lets you pick the active modulation and configure its own parameters (e.g. spreading factor, bandwidth and coding rate for LoRa; bitrate and deviation for GFSK; sync words and coding rate for FLRC; etc.). Only the modulations supported by your connected shield are selectable — for instance FLRC is only available on LR2021.

You can also, like on the connexion page, identify and reset the connected kit using the and buttons.

The configuration can be saved, loaded and even copied and pasted easily using the toolbar.

Toolbar

On both the [Radio Configuration](#) and [Application](#) pages, a tool bar allows you to :



-  Save the configuration to a JSON file. This will open a dialog to select the location and name of the file.
-  Load a configuration from a file.
-  Copy the configuration to the clipboard: JSON content can be pasted to wherever you would like or...
-  ... paste it on the page of another connected kit to ensure they both use the exact same parameters.
-  Update the firmware of the connected kit. This will open a dialog to select the firmware file. The firmware file must be a .bin, .hex, or .elf file and correspond to your kit.
- An "NVM Reset" button that allows you to reset the NVM of your kit. This will erase all the configurations and logs stored in the NVM and reset it to its default state. This is useful in case of any issue with the kit or if you want to start fresh. Please make sure to properly stop any running application before using this button to avoid any issue. Also, use it with caution as it will erase all the data stored in the NVM without any confirmation.
- A standalone mode button to switch between connected and standalone mode when running the demonstration applications in standalone mode. More details about this button and the standalone mode in the [Running demonstration application](#) section below.
- A "Disconnect" button allows you to disconnect the board from the software. After clicking on this, the device will be back as available for connection on the welcome page.

Once you are satisfied with your configuration, you can go to the [Application](#) page by clicking on the icon in the left menu.

From this page, you can select, set parameters and run demonstration applications.

Running demonstration application

From the [Application](#) page, you can select and run different demonstration applications. The available applications are:

- Continuous Wave
- Ping Pong
- PER (Packet Error Rate) Test
- Static modes for current consumption measurement

While an application that receives packets (e.g. Ping Pong, PER) is running, its **Counter**, **RSSI**, and **FEI** (Frequency Error Indicator) are shown live next to the Start/Stop buttons. **FEI** is smoothed over the last 7 values and reads *N/A* when it is not available for the current modulation.

Next to these buttons, a **"Restart with radio frequency adjust"** button lets you correct for a measured frequency error in one click: it takes the last received FEI value, adds it to the current RF frequency, saves and applies the corrected configuration, then stops and restarts the running application with it. This button has no effect if no FEI value has been received yet.

The screenshot displays the LoRa application interface. At the top, a status bar shows the device path `/dev/ttyACM0`, a battery icon, and buttons for `NVM Reset`, `Standalone mode` (selected), and `Disconnect`. Below this is the **Applications** section with tabs for `Continuous Wave`, `Packet Error Rate` (selected), `Ping Pong`, and `Current Consumption`. The **Packet Error Rate** configuration area includes input fields for `Number of frames` (20), `Delay between transmissions (ms)` (200), and `Reception timeout (ms)` (1000). A `Mode` dropdown is set to `Receiver`. To the right are buttons for `Start`, `Stop`, and `Restart with radio frequency adjust`. Below the configuration is a **Logs** section with a scrollable log area showing system and application messages. At the bottom, a footer displays `v1.7.3`, `Firmware version: v1.7.0-44-ga4c8a03ef`, `Chip version: LR2021EVK1XB51_XIAO`, and `Connection status: connected`.

Applications

Continuous Wave Packet Error Rate Ping Pong Current Consumption

▼ **Packet Error Rate**

Number of frames Delay between transmissions (ms) Reception timeout (ms)

20 200 1000

Mode

Receiver

Counter RSSI (dBm) FEI (Hz)

Timeout Timeout Timeout

Start

Stop

Restart with radio frequency adjust

Logs

```
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - LoRa modulation parameters:
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Spreading factor = SF7
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Bandwidth = 125_KHZ
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Coding rate = 4_5
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - LDRO = Disabled
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - LoRa packet parameters:
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Preamble length = 8 symbol(s)
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Header mode = EXPLICIT
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Payload length = 7 byte(s)
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - CRC mode = Enabled
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - IQ = Inverted
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - LoRa syncword = 0x12
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Interrupt flags = 0x00000008
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Interrupt flags (after filtering) = 0x00000008
[*] - [/dev/ttyACM0] - [WARNING] - [APPLICATION] - Rx timeout
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Waiting for first frame...
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Interrupt flags = 0x00000008
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Interrupt flags (after filtering) = 0x00000008
[*] - [/dev/ttyACM0] - [WARNING] - [APPLICATION] - Rx timeout
[*] - [/dev/ttyACM0] - [INFO] - [APPLICATION] - Waiting for first frame...
```

v1.7.3 Firmware version: v1.7.0-44-ga4c8a03ef Chip version: LR2021EVK1XB51_XIAO Connection status: connected

In the screenshot above, no packet has been received yet (no peer kit was transmitting), so Counter/RSSI/FEI show "Timeout"; once packets are received these fields show live numeric values.

Each application has its own parameters that are set while starting it. You can run the application while connected to the computer or in standalone mode powering the kit(s) by power banks or batteries.

Connected mode

In connected mode, the kit is powered by the USB connection to your computer and the application running on your computer controls the kit. The logs of the application are displayed in the log area of the application and some basic info is shown on the OLED screen (if available).

To run an application in connected mode, you need to:

1. Select your kit(s) on the connection page and click on the Connect button, the application loads and opens automatically the [Radio Configuration](#) page.
2. Configure the radio parameters as desired

3. Go to the [Application](#) page
4. Configure the application parameters as desired
5. Click on the [Start](#) button to start the application
6. Stop the application, by clicking on the [Stop](#) button

You can also save the logs to a text file by clicking on the  button above the log area.

Standalone mode

In standalone mode, the kit(s) is/are powered by an external power source (power bank, batteries, etc.) and the application running on the kit controls the demo. Info about the status of the application is displayed on the OLED screen of the kit(s) (if available) and the logs are stored in the NVM of the kit(s).

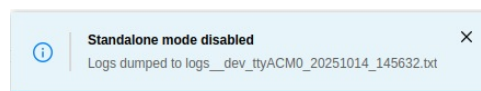
To run an application in standalone mode, you need to:

1. Plug the kit to your computer
2. Select your kit on the connection page and click on the Connect button, the application loads and opens automatically the [Radio Configuration](#) page.
3. Configure the radio parameters and the application parameters as desired.
4. Click on the [Start](#) button to start the application.
5. Stop the application, by clicking on the [Stop](#) button. This step is mandatory to properly save the configuration and application parameters in the NVM of the kit.
6. Set the device in standalone mode by sliding the standalone button to the right. The inner button will turn green and the configurations fields will be disabled (greyed) to indicate that the kit is now in standalone mode:



The kit is now ready to run in standalone mode and will stay in this mode until you deactivate it.

7. Unplug the kit from the computer and power it with an external power source and close LoRaStudio
8. To start the application in standalone mode, press the user button on the kit
9. Make sure to stop the application in standalone mode by pressing the user button on the kit again
10. Connect back the kit to your computer.
11. Open again LoRa Studio, go back to the connection page, search for devices, select and connect your kit.
12. To retrieve the logs of the application, go to the Application page and deactivate the standalone mode. The inner button will turn grey and the configuration fields will be enabled again to indicate that the kit is now back in connected mode. Also a toast will appear to indicate that the logs have been retrieved and saved to a text file with its name in the LoRa Studio folder:



It also opens it for you automatically using your default text editor.

Application details

Continuous Wave Demo

The Continuous Wave demo allows you to transmit a continuous wave signal for testing purposes.

This application can be used to test the range of the radio, the sensitivity of a receiver or to perform regulatory tests.

It does not have any application specific parameters. You only need to configure the radio parameters. Just use the [Start](#) button to start the transmission and the [Stop](#) button to stop it.

Ping Pong Demo

The Ping Pong demo demonstrates bi-directional communication between two devices. This application obviously needs two kits to work with one another. *TIP: Configure only one kit properly then use the copy/paste   configuration functionality to make sure you are running with the same parameters on both kits.*

The Ping Pong demo requires the following application parameters:

- Delay before transmission: set the delay before the first packet is sent in ms
- Delay between packets: set the delay in ms between the reception and the retransmission of the packet
- Sync packet threshold: Number of packets to transmit without a response to give enough margin for synchronization
- Prefix size: Number of bytes used as prefix in the packet
- Reception timeout: Time in ms to wait for a response before considering that the packet is lost

When starting the application, the kit will start transmitting packets and wait for a response. If a response is received, it will take its content (a counter) and increase its value before retransmitting the packet after the configured delay. If no response is received within the timeout, it will consider the packet lost and transmit a new one after the configured delay and put itself in reception mode again.

Since no actual synchronization is done between the two kits, it is possible that both kits transmit at the same time and therefore miss each other's packets. The synchronization threshold allows to mitigate this issue by transmitting a certain number of packets without waiting for a response. This increases the chances that one of the packets will be received by the other kit and that the communication can start.

Packet Error Rate

The Packet Error Rate evaluates link quality and performance metrics. This application needs two kits to work with one set as transmitter and the other as receiver. *TIP: Configure only one kit properly then use the copy/paste   configuration functionality and change the mode on the second kit afterwards.*

Select Packet Error Rate and setup the application parameters:

- Number of frames: Number of frames to receive (as goal) for the reception mode, on which the PER will be calculated. If set to 0, the kit will receive indefinitely and PER will be calculated on the cumulated packets
- Delay between transmissions: Delay in ms between each frame the transmitter sends
- Reception timeout: Time in ms to wait before considering a frame lost
- Mode: Select if the kit will be a transmitter or receiver

A live calculated PER will be shown in the logs and on the OLED screen for you to see along with the number of received packets.

Static modes for current consumption measurement

This application allows you to set the kit in different static modes to measure the current consumption.

1. Once connected to your kit, the application loads and opens automatically the [Radio Configuration](#) page.
2. Configure the radio parameters (frequency, power, modulation...)
3. Go to the [Application](#) page
4. Select Static modes and setup the application parameters:
 - Mode: Select the static mode you want to set the kit in
5. Click on the **Start** button to set the kit in the selected static mode
6. The kit will enter the selected static mode
7. Click on the **Stop** button to exit the static mode and return to normal operation

Firmware Update

To update the firmware of your kit, you need to connect it to your computer and open LoRa Studio. Then, on either the radio configuration page or the application page, click on the  button in the toolbar. This will open a file dialog to select the firmware file. The application will automatically open the folder where the firmwares can be found. Choose the file that corresponds to your kit (.bin, .hex, or .elf).

Once the firmware file is selected, the update process will start automatically. A progress bar will show the progress of the update. Do not disconnect or turn off the kit during the update process.

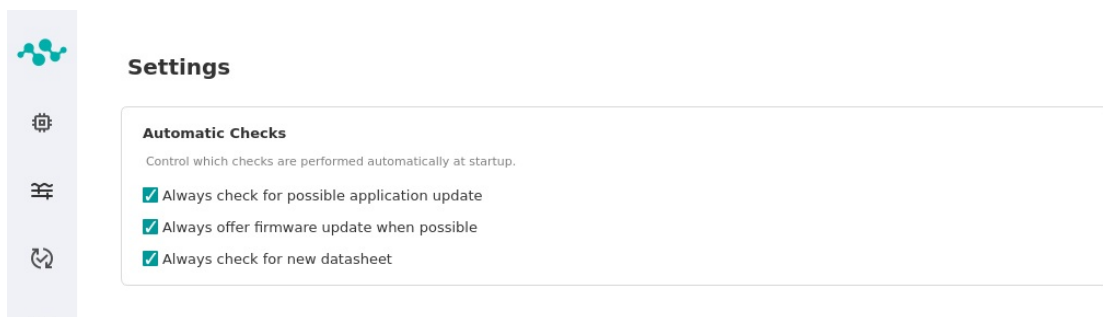
Once the update is complete, a message will indicate that the update was successful. You can now use the new firmware.

Settings page

The  **Settings** page, reached from the left navigation menu, groups the automatic checks LoRa Studio performs at startup, under **Automatic Checks**:

- **Always check for possible application update:** check for a newer LoRa Studio release at startup.
- **Always offer firmware update when possible:** prompt to update the kit's firmware when a newer one is available.
- **Always check for new datasheet:** check for a newer revision of the radio datasheet at startup (see the datasheet note in the [Radio Configuration](#) section above). When a newer revision is found, a dialog compares the current and available revisions and lets you download the latest one.

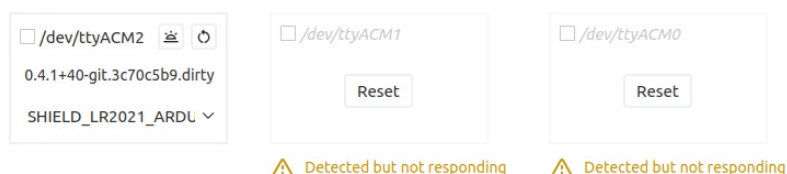
Each of these checks can be switched off individually if you prefer to manage updates manually.



Troubleshooting

Common connection issues

Main page shows a device is "Detected but not responding":



This usually means that the application can see a serial port to communicate with the device, but the device is not responding to commands. It can be that this device is not a supported development kit or the firmware on the kit is not compatible with LoRa Studio. It can also be that the kit has some UART issues. You can try the following steps to resolve the issue:

- Click on the **Search for devices** button.
- Click on the "Reset" button associated to the serial port you believe your board is connected to and then on the "Search" button again.
- Physically reset the kit by pressing the reset button and then click on the "Search for devices" button again.

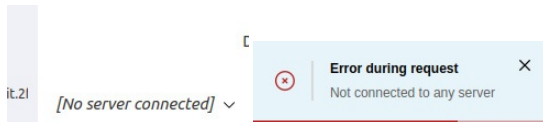
If none of the above steps work, you can try to restart the Backend by following the [Restart the backend](#) section below

Note: For nRF54L15-DK based kits, even though you made sure that you have deactivated the first serial port in nRF Connect, two serial ports will be detected, and one of them will always be not responsive. This does not affect the functionality of LoRa Studio and can be ignored.

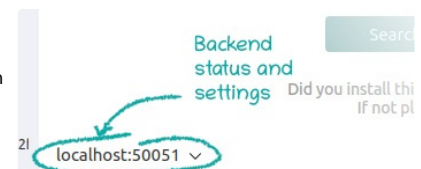
No device shows at all

- Verify driver installation: Double-check that all necessary drivers are installed and up to date.
- Restart the application: Sometimes, simply restarting LoRa Studio can resolve connection issues.
- Try another USB port or cable: Faulty USB ports or cables can cause connection problems. Switch to a different port or use a different cable to see if that resolves the issue.
- Check device power: Ensure that the development kit is powered on and functioning correctly.
- Check for conflicts: Ensure that no other applications are using the same serial port that LoRa Studio is trying to access.

No server connection

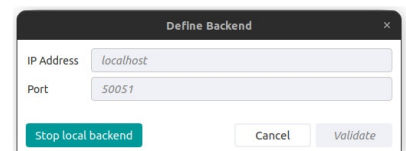


It can happen that the backend does not start properly and hence no device can be connected. You can verify and fix this issue by clicking on the server info at the bottom left of the connexion page.



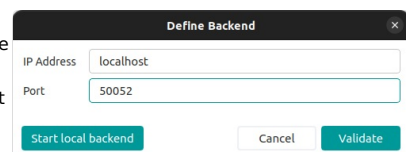
Restart the backend

A dialog box will open, click on the **Stop local backend** button. Some toast will indicate the stopping of the backend, the button shall change into **Start local backend**. Click on this button again, this shall restart the backend properly.



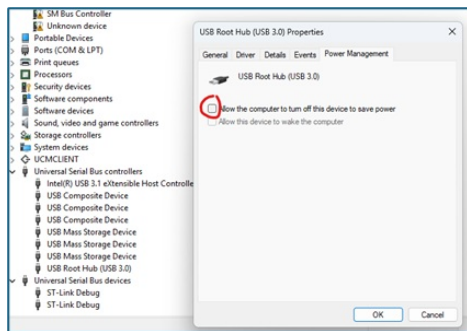
Changing the port

It can also happen that another RPC server is running on your machine and is using the same port as the LoRa Studio backend, which prevents it from starting properly. In this case, you can change the port used by the LoRa Studio backend by stopping the server as explained above and then changing the port number in the input field before restarting it.



Kit not responding after some commands

If the kit stops responding after some commands, it could be due to a power saving policy of your operating system that puts the USB port to sleep. You can try to disable this feature in your OS settings. It is known to have happened on Windows 11. Simply untick the option "Allow the computer to turn off this device to save power" for the USB Root Hubs in the Device Manager as shown below:



Command-Line Interface (CLI)

In addition to the graphical interface, LoRa Studio ships a command-line front end, `LoRaStudioCli`, to control Semtech radio dev kits from a terminal or from scripts. Like the GUI, the CLI is a client: it connects to the **host backend** over gRPC, and the backend in turn talks to the embedded firmware running on the connected dev kit.



Note: Because of this architecture, **the backend host must be running before the CLI can do anything useful**. All the binaries mentioned below (`LoRaStudioHost` and `LoRaStudioCli`) are located in the LoRa Studio installation folder.

Starting the host backend

Open a terminal and start the host:

```
LoRaStudioHost
```

Copy

You can control the log verbosity and the listening address:

```
LoRaStudioHost --help
# usage: LoRaStudioHost [-h] [--version] [-i IP_ADDRESS]
#                               [-l {DEBUG,INFO,WARNING,ERROR,CRITICAL}] | --log-config LOG_CONFIG

LoRaStudioHost -l INFO          # run with INFO logging
LoRaStudioHost -i '[:]:50051'   # change listening address (default)
```

Typical startup output (a dev kit plugged on `/dev/ttyACM0` was auto-detected):

```
INFO - Starting GRPC server (v.1.7.3)
INFO - Server IP: [:]:50051
INFO - Adding new device(s) ['/dev/ttyACM0']...
INFO - Device hotplug monitor: using pyudev (event-driven)
```

The backend listens on `localhost:50051` by default and **detects devices automatically** (including hotplug — plug/unplug while it runs). Leave this terminal open and use a **second terminal** for the CLI.

CLI basics

```
LoRaStudioCli [OPTIONS] COMMAND [ARGS]...
```

Global options

Option	Default	Description
<code>-u, --server-url</code>	<code>localhost:50051</code>	URL of the backend server to connect to
<code>--version</code>	—	Print the CLI version and exit
<code>--help</code>	—	Show help (works on every command and subgroup)

```
LoRaStudioCli --version
# LoRaStudioCli, version 1.7.3

LoRaStudioCli --help          # top-level help
LoRaStudioCli command --help  # help for the "command" subgroup
LoRaStudioCli command DEV startdemo --help # help for a specific subcommand
```

Note: `--version` reflects the version the CLI package was built from (its git tag); it will differ depending on which release you installed.

If the backend is on another machine or port, point the CLI at it:

```
LoRaStudioCli -u 192.168.1.50:50051 list-devices
```

Copy

Note: Most operations target a specific device, identified by its **device name** as reported by `list-devices` (on Linux this is a serial port path such as `/dev/ttyACM0`).

Top-level commands

These are invoked directly: `LoRaStudioCli <command> ...`

list-devices

List the devices the backend currently knows about.

```
LoRaStudioCli list-devices
# 0: /dev/ttyACM0

LoRaStudioCli list-devices -v      # also print firmware/version information
# 0: /dev/ttyACM0
# {'build_date': '2026-09-14T14:29:28+02:00',
#  'code_version_date': '2026-09-14T14:11:47+02:00',
#  'code_version_tag': 'v1.7.3-16-gc16cf15d4', 'is_official': True}
```

Copy

Option	Description
-v, --verbose	Also show each device's firmware version

backend-version

Print the version string of the running backend.

```
LoRaStudioCli backend-version
# 1.7.3
```

Copy

hard-reset DEVICE_NAME

Hard-reset a device.

```
LoRaStudioCli hard-reset /dev/ttyACM0
```

Copy

flashfirmware DEVICE_NAME BINARY_FILE_PATH

Flash a firmware binary to the device. Progress is reported as a percentage. `BINARY_FILE_PATH` accepts `.bin`, `.hex`, or `.elf` files — the format is auto-detected from the extension.

```
LoRaStudioCli flashfirmware /dev/ttyACM0 ./LoRaStudio_xxx_vvv.bin
# Flashing /dev/ttyACM0: 0%
# Flashing /dev/ttyACM0: 50%
# Flashing /dev/ttyACM0: 100%
```

Copy

The command group

All device-control operations live under the `command` group. The device name (and optional tag) are given **once, on the group**, before the subcommand:

```
LoRaStudioCli command [OPTIONS] DEVICE_NAME <subcommand> [ARGS]...
```

Copy

Option	Default	Description
-t, --tag	0	Datagram tag used for the command

Example shape:

```
LoRaStudioCli command /dev/ttyACM0 identify
LoRaStudioCli command -t 3 /dev/ttyACM0 startdemo DEMO_PING_PONG
```

[Copy](#)

Device information

Subcommand	Description	Example output
identify	Make the device identify itself	True
gettargetboard	Get the board name	NUCLEO_L476RG
getshield	Get the currently configured shield	SHIELD_LR1110MB1DJS
getsupportedshields	List shields the device supports	['SHIELD_LR2021_ARDUINO_LEGACY', ...]
getrunningmode	Get the device running mode	RUNNING_MODE_CONNECTED
getlogsize	Get the size of saved logs (bytes)	4294965256

```
LoRaStudioCli command /dev/ttyACM0 identify
LoRaStudioCli command /dev/ttyACM0 gettargetboard
LoRaStudioCli command /dev/ttyACM0 getshield
LoRaStudioCli command /dev/ttyACM0 getsupportedshields
LoRaStudioCli command /dev/ttyACM0 getrunningmode
LoRaStudioCli command /dev/ttyACM0 getlogsize
```

[Copy](#)

Shield and running mode

```
LoRaStudioCli command /dev/ttyACM0 setshield SHIELD_LR1110MB1DJS
LoRaStudioCli command /dev/ttyACM0 setrunningmode RUNNING_MODE_STANDALONE
```

[Copy](#)

`setshield` and `setrunningmode` validate their argument; if the value is unknown the CLI prints the list of valid values and exits with an error. See the [value reference](#) below.

Demos

```
LoRaStudioCli command /dev/ttyACM0 startdemo DEMO_PING_PONG
LoRaStudioCli command /dev/ttyACM0 stopdemo
```

[Copy](#)

`startdemo DEMO` starts a demo; `DEMO` must be one of the [demo values](#). `stopdemo` stops the running demo.

Buttons & live telemetry

```
LoRaStudioCli command /dev/ttyACM0 pushbutton      # emulate a button press
LoRaStudioCli command /dev/ttyACM0 logs             # stream device logs (colored)
LoRaStudioCli command /dev/ttyACM0 rxpacketstatus  # stream Rx packet status
```

[Copy](#)

- `logs` streams log lines as `[device]:[level] - message`, color-coded by level and source. It runs until interrupted (`Ctrl-C`). The active FLRC configuration parameters are printed here whenever the configuration is set.
- `rxpacketstatus` streams received-packet statistics: `[device]:[pkt : <counter> - RSSI : <rss> - FEI : <fei> - Status <status>]`. FEI is a median of the last 7 values and is shown as `N/A` when it is not available for the current modulation or when the raw value is rejected as an outlier (beyond ± 50 kHz). FEI is now also reported for FLRC, not just LoRa/FSK.

Non-volatile memory (NVM)

```
LoRaStudioCli command /dev/ttyACM0 storeconfig DEMO_PER SHIELD_LR1110MB1DJS # save current config to NVM
LoRaStudioCli command /dev/ttyACM0 loadconfigfromnvm                        # load config from NVM
LoRaStudioCli command /dev/ttyACM0 resetnvm                                # erase NVM
LoRaStudioCli command /dev/ttyACM0 getlogsize                              # size of logs in NVM
LoRaStudioCli command /dev/ttyACM0 dumplogsfromnvm logs.txt                 # dump NVM logs to a file
```

[Copy](#)

- `storeconfig DEMO SHIELD_NAME` stores the current configuration in NVM, tagged with the given demo and shield (both validated against the lists below).
- `dumplogsfromnvm LOG_FILE` writes NVM logs to `LOG_FILE`, one line per entry as `[source]:[device]:[level] - message`.

Configuration (get / set)

The device configuration is a JSON document with two top-level sections, `appli_configuration` (per-demo application settings) and `radio_configuration` (modem/PHY settings such as FSK, FLRC, LoRa parameters, PA, etc.).

Read the configuration into a file:

```
LoRaStudioCli command /dev/ttyACM0 getconfiguration config.json
```

Copy

Write a configuration back:

```
LoRaStudioCli command /dev/ttyACM0 sendconfiguration config.json
```

Copy

Option	Description
<code>-f, --force</code>	Send the file as-is instead of merging it onto the device's config

Behaviour of `sendconfiguration`:

- **Without `-f` (default, recommended):** the CLI first fetches the current configuration from the device, **merges** the keys present in your file onto it, and sends the result. This lets you edit just a few fields. If your file contains a key that does not exist on the device, the command reports the offending key and exits with an error.
- **With `-f/--force`:** the file is sent verbatim, with no merge. The file must be a complete, valid configuration.

Typical edit workflow:

```
# 1. Pull the current config
LoRaStudioCli command /dev/ttyACM0 getconfiguration config.json
# 2. Edit config.json (change only the fields you need)
# 3. Push it back (merged onto the device config)
LoRaStudioCli command /dev/ttyACM0 sendconfiguration config.json
```

Copy

Excerpt of a configuration file:

```
{
  "appli_configuration": {
    "per": { "nb_frames": 20, "receiver": false, "rx_timeout_in_ms": 1000 },
    "ping_pong": { "delay_before_tx": 200, "rx_timeout": 1000 }
  },
  "radio_configuration": {
    "enable_rx_boost_mode": true,
    "fallback_mode": "FALLBACK_STDBY_RC",
    "fsk_parameters": { "fsk_bitrate": 50000, "fsk_dev": 25000 },
    "rf_path": "AUTO_PATH",
    "xtal_tcxo_config": { "has_tcxo": true, "tcxo_config": "TCXO_CTRL_3_3V" }
  }
}
```

New `radio_configuration` fields:

Field	Type / values	Description
<code>rf_path</code>	<code>RFPath</code> , see value reference	Select which RF path the board uses
<code>xtal_tcxo_config.has_tcxo</code>	<code>bool</code>	Whether the shield has a TCXO (vs. XTAL)
<code>xtal_tcxo_config.tcxo_config</code>	<code>TcxoVoltage</code> , see value reference	TCXO supply voltage, when <code>has_tcxo</code> is <code>true</code>
<code>lora_parameters.lora_cr</code>	<code>LoraCr</code> , now also <code>LORA_CR_LI_CONVOLUTIONAL_4_6</code> / <code>LORA_CR_LI_CONVOLUTIONAL_4_8</code>	New convolutional coding rates for LoRa

Command quick reference

Command	Description
<code>list-devices [-v]</code>	List known devices
<code>backend-version</code>	Backend version
<code>hard-reset DEVICE</code>	Hard reset a device

flashfirmware DEVICE BIN	Flash firmware
command [-t TAG] DEVICE identify	Identify the device
command DEVICE gettargetboard	Get board name
command DEVICE getshield	Get current shield
command DEVICE setshield SHIELD	Set shield
command DEVICE getsupportedshields	List supported shields
command DEVICE getrunningmode	Get running mode
command DEVICE setrunningmode MODE	Set running mode
command DEVICE startdemo DEMO	Start a demo
command DEVICE stopdemo	Stop the running demo
command DEVICE pushbutton	Emulate a button press
command DEVICE logs	Stream device logs
command DEVICE rxpacketstatus	Stream Rx packet status
command DEVICE getconfiguration FILE	Read config to file
command DEVICE sendconfiguration [-f] FILE	Write config from file
command DEVICE storeconfig DEMO SHIELD	Store config in NVM
command DEVICE loadconfigfromnvm	Load config from NVM
command DEVICE resetnvm	Erase NVM
command DEVICE getlogsize	Size of saved logs
command DEVICE dumplogsfromnvm FILE	Dump NVM logs to file

Value reference

The *Demos*, *Running modes* and *Shields* enumerations below are validated by the CLI: passing an invalid value prints the list of accepted values and exits with code 1. *RF path* and *TCXO voltage* are used inside the JSON configuration and are not validated by the CLI itself — an invalid value is rejected by the device when the configuration is applied.

Demos

Used by startdemo and storeconfig:

- DEMO_TX_CW
- DEMO_PER
- DEMO_EXAMPLE
- DEMO_STATIC_MODE
- DEMO_PING_PONG

Running modes

Used by setrunningmode:

- RUNNING_MODE_CONNECTED
- RUNNING_MODE_STANDALONE

Shields

Used by setshield and storeconfig (use getsupportedshields to see which ones your specific device accepts):

SHIELD_LR1110MB1DIS	SHIELD_LR1110MB1DJS	SHIELD_LR1110MB1GIS	SHIELD_LR1110MB1GJS
SHIELD_LR1120MB1DIS	SHIELD_LR1120MB1DJS	SHIELD_LR1120MB1GIS	SHIELD_LR1120MB1GJS
SHIELD_LR1121MB1DIS	SHIELD_LR1121MB1GIS		
SHIELD_LR2021_ARDUINO_LEGACY			
SHIELD_LR2021EVK1XBS1_ARDUINO	SHIELD_LR2021EVK1XCS1_ARDUINO		SHIELD_LR2021EVK1XGS1_ARDUINO
SHIELD_LR2021EVK1XBS1_XIAO	SHIELD_LR2021EVK1XCS1_XIAO		SHIELD_LR2021EVK1XGS1_XIAO
SHIELD_LR2022EVK1XBS1_ARDUINO	SHIELD_LR2022EVK1XCS1_ARDUINO		SHIELD_LR2022EVK1XGS1_ARDUINO
SHIELD_LR2022EVK1XBS1_XIAO	SHIELD_LR2022EVK1XCS1_XIAO		SHIELD_LR2022EVK1XGS1_XIAO
SHIELD_LR2012EVK1XBS1_ARDUINO	SHIELD_LR2012EVK1XCS1_ARDUINO		SHIELD_LR2012EVK1XGS1_ARDUINO
SHIELD_LR2012EVK1XBS1_XIAO	SHIELD_LR2012EVK1XCS1_XIAO		SHIELD_LR2012EVK1XGS1_XIAO
SHIELD_RADIOLESS			

RF path

Used by the `rf_path` field of `radio_configuration`:

- `AUTO_PATH`
- `LF_PATH`
- `HF_PATH`

TCXO voltage

Used by the `xtal_tcxo_config.tcxo_config` field of `radio_configuration`, when `xtal_tcxo_config.has_tcxo` is true:

- `TCXO_CTRL_1_6V`
- `TCXO_CTRL_1_7V`
- `TCXO_CTRL_1_8V`
- `TCXO_CTRL_2_2V`
- `TCXO_CTRL_2_4V`
- `TCXO_CTRL_2_7V`
- `TCXO_CTRL_3_0V`
- `TCXO_CTRL_3_3V`

CLI troubleshooting

Symptom	Likely cause / fix
<code>Error: ... connection refused / unavailable</code>	The backend is not running, or is on another host/port. Start <code>LoRaStudioHost</code> , or pass <code>-u HOST:PORT</code> .
<code>list-devices</code> shows nothing	No dev kit is connected, or it has not enumerated yet. Plug it in (the backend hotplug-detects it) and retry. On Linux check that the serial port (e.g. <code>/dev/ttyACM0</code>) exists and you have permission to access it.
<code>Demo '...' does not exist / Shield named '...' does not exist / Running mode '...' does not exist</code>	The enum value is misspelled. Use a value from the value reference .
Should be encodable on one byte	<code>--tag</code> must be between 0 and 255.
<code>non-hexadecimal number found on send-raw-message</code>	DATA must be a valid hex string (even number of hex digits).
Firmware flashing fails	Ensure the <code>pyocd Nucleo</code> pack is installed: <code>pyocd pack install stm32l476rg</code> .

For any command, append `--help` to see its exact usage, arguments and options.

Annex A — Configuration Parameter Reference

This annex lists every parameter of the device configuration, together with its possible values and the input ranges accepted by LoRa Studio. The configuration is the document you read with `getconfiguration` / Save and write back with `sendconfiguration` / Load. It has two top-level sections:

- `appli_configuration` — per-application (demo) settings.
- `radio_configuration` — modem / PHY settings.

How values are encoded in the file:

- **Enumerations** are written as their exact upper-case name, e.g. `"lora_sf": "SF7"`.
- **Integers** are always written in **decimal**, even for fields the GUI displays in hexadecimal or binary. For example a LoRa sync word shown as `0x12` in the GUI is stored as `"lora_sync_word": 18`. The *Format* column below shows how the GUI presents the value (DEC / HEX / BIN).
- **Booleans** are written as `true` / `false`.

Shield / radio family: some parameters, their available choices and their ranges depend on the radio of the connected shield. Throughout this annex:

- **LR11xx** = LR1110 / LR1120 / LR1121 shields (`SHIELD_LR11xxMB1...`).
- **LR20xx** = LR2021 / LR2022 / LR2012 shields (`SHIELD_LR20xxEVK...` and `SHIELD_LR2021_ARDUINO_LEGACY`).

The configuration object always contains every section (e.g. both `lr11xx_pa_config` and `lr20xx_pa_config`, and both ramp-up-time fields); only the ones matching the connected radio take effect. Ranges shown are those validated by LoRa Studio; the firmware ultimately enforces what is physically achievable for the selected radio and shield.

A.1 Application parameters (`appli_configuration`)

Example (example)

Key	Type	Range / values	Unit	Notes
<code>delay</code>	integer	20 - 1000	ms	Inter-packet delay.

Packet Error Rate (`per`)

Key	Type	Range / values	Unit	Notes
<code>nb_frames</code>	integer	0 - 1000	—	Number of frames; 0 = receive indefinitely.
<code>tx_to_tx_delay_in_ms</code>	integer	1 - 999	ms	Delay between transmissions.
<code>rx_timeout_in_ms</code>	integer	100 - 30000	ms	Reception timeout.
<code>receiver</code>	boolean	<code>true</code> (Receiver) / <code>false</code> (Transmitter)	—	Role of the device.

Ping Pong (`ping_pong`)

Key	Type	Range / values	Unit	Notes
<code>delay_before_tx</code>	integer	0 - 999	ms	Delay before first transmission.
<code>delay_ping_pong_pace</code>	integer	0 - 999	ms	Delay between ping and pong.
<code>sync_packet_threshold</code>	integer	0 - 255	—	Max packets to try sync before giving up.
<code>prefix_size</code>	integer	0 - 245	bytes	Prefix size in the packet.
<code>rx_timeout</code>	integer	0 - 4096	ms	Reception timeout.

Static / current-consumption mode (`static_mode`)

Key	Type	Possible values
<code>static_mode</code>	enum	NOT_CONFIGURED, SLEEP_WITH_RETENTION, SLEEP_WITHOUT_RETENTION, SLEEP_WITH_RETENTION_WITH_RTC, SLEEP_WITHOUT_RETENTION_WITH_RTC, STANDBY_RC, STANDBY_XOSC, RX, TX, FS

A.2 Common radio parameters (`radio_configuration`)

Key	Type	Range / values	Unit	Notes
<code>rf_freq_in_hz</code>	integer	150 000 000 - 2 500 000 000	Hz	Each shield is optimised for a sub-band (e.g. 863-870 MHz); stay within the shield's band for good performance.

tx_output_pwr_in_half_dbm	integer (half-dBm)	-34 – 44 (= -17.0 – +22.0 dBm)	½ dBm	Value is in half-dBm steps : 44 = +22 dBm, -34 = -17 dBm. Max depends on radio/shield.
enable_rx_boost_mode	boolean	true / false	—	Reception boost mode.
payload_length	integer	7 – 255 (LR11xx, LR2012) 7 – 511 (LR2021, LR2022)	bytes	Used for fixed-length / implicit modes.
packet_type	enum	PKT_TYPE_LORA, PKT_TYPE_GFSK, PKT_TYPE_OOK, PKT_TYPE_FLRC	—	Active modulation. PKT_TYPE_FLRC is selectable on LR2021 only .
fallback_mode	enum	FALLBACK_STDBY_RC, FALLBACK_STDBY_XOSC, FALLBACK_FS	—	State entered after Tx/Rx.
regulator_mode	enum	SIMO_OFF, SIMO_NORMAL	—	SIMO regulator. (RFU, RFU_2 exist in the protocol but are reserved.)
lr11xx_ramp_up_time	enum	LR11XX_RADIO_RAMP_16_US ... LR11XX_RADIO_RAMP_304_US (16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 240, 272, 304 µs)	µs	PA ramp time, LR11xx radios. Default 48 µs.
lr20xx_ramp_up_time	enum	LR20XX_RADIO_COMMON_RAMP_2_US ... LR20XX_RADIO_COMMON_RAMP_304_US (2, 4, 8, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 240, 272, 304 µs)	µs	PA ramp time, LR20xx radios. Default 48 µs.
rf_path	enum	AUTO_PATH, LF_PATH, HF_PATH	—	Selects which RF path the board uses.
xtal_tcxo_config.has_tcxo	boolean	true / false	—	Whether the shield has a TCXO (vs. XTAL).
xtal_tcxo_config.tcxo_config	enum	TCXO_CTRL_1_6V, _1_7V, _1_8V, _2_2V, _2_4V, _2_7V, _3_0V, _3_3V	—	TCXO supply voltage, used when has_tcxo is true.

Capacitor trim (capacitor_trim)

Key	Type	Range / values	Format	Notes
is_configured	boolean	true / false	—	When false, the trim fields are ignored.
xta	integer	0 – 47	HEX	XTA trim, value as per datasheet.
xtb	integer	0 – 47	HEX	XTB trim, value as per datasheet.
startup_time_us	integer	0 – 255	DEC	Optional extra XTAL startup delay (µs). Default 0.

Power amplifier — LR20xx (lr20xx_pa_config)

Key	Type	Range / values	Notes
is_configured	boolean	true / false	When false, power is driven by tx_output_pwr_in_half_dbm instead.
tx_power	integer	-39 – 44	Raw PA power register, value as per datasheet.
pa_sel	enum	PA_PATH_LF, PA_PATH_HF	Low-frequency or high-frequency PA path.
pa_lf_duty_cycle	integer	0 – 47	LF PA duty cycle, value as per datasheet.
pa_lf_slices	integer	0 – 15	LF PA slices, value as per datasheet.
pa_hf_duty_cycle	integer	0 – 47	HF PA duty cycle, value as per datasheet.

Power amplifier — LR11xx (lr11xx_pa_config)

Advanced PA register configuration for **LR11xx** radios. LoRa Studio normally drives LR11xx power through tx_output_pwr_in_half_dbm; set these only if you need raw register control. All values are as per the radio datasheet.

Key	Type	Values	Notes
is_configured	boolean	true / false	When false, fields are ignored.
tx_power	integer	per datasheet	Raw PA power register.
pa_sel	enum	PA_PATH_LF, PA_PATH_HF	PA path selection.
pa_reg_supply	integer	per datasheet	PA regulator supply.
pa_duty_cycle	integer	per datasheet	PA duty cycle.
pa_hp_sel	integer	per datasheet	High-power PA selection.

A.3 LoRa parameters (lora_parameters)

Key	Type	Range / possible values	Unit / Fmt	Notes
lora_sf	enum	SF5, SF6, SF7, SF8, SF9, SF10, SF11, SF12	—	Spreading factor.
lora_bw	enum	LORA_BW_007_KHZ, _010, _015, _020, _031, _041, _062, _083, _101, _125, _200, _250, _400, _500, _800, _1000, _1600_KHZ	—	Bandwidth. The GUI exposes 31 kHz–1000 kHz; the other values are valid but uncommon.
lora_cr	enum	LORA_CR_4_5, _4_6, _4_7, _4_8, LORA_CR_LI_4_5, _LI_4_6, _LI_4_8, LORA_CR_LI_CONVOLUTIONAL_4_6, _CONVOLUTIONAL_4_8	—	Coding rate. The two CONVOLUTIONAL rates are available on LR20xx only .
lora_ldro	enum	LORA_LDRO_OFF, LORA_LDRO_ON, LORA_LDRO_AUTO	—	Low data-rate optimization.
lora_pkt_mode	enum	LORA_PKT_EXPLICIT, LORA_PKT_IMPLICIT	—	Header mode.
lora_sync_word	integer	1 - 4 294 967 295	HEX	Public LoRaWAN = 0x34 (52); private LoRa = 0x12 (18).
lora_preamble_len	integer	6 - 65536	symbols	Preamble length.
lora_iq	boolean	true (Inverted) / false (Standard)	—	IQ polarity.
lora_crc	boolean	true / false	—	CRC on/off.

A.4 FSK / GFSK parameters (fsk_parameters)

Key	Type	Range / possible values	Unit / Fmt
fsk_dev	integer	0 - 16 777 215	Hz
fsk_bitrate	integer	0 - 4 294 967 295	bps
fsk_bandwidth	integer	3 500 - 3 076 000	Hz
fsk_pulse_shape	enum	GFSK_PULSE_SHAPE_OFF, _BT_03, _BT_05, _BT_07, _BT_1	—
fsk_preamble_len	integer	0 - 65 535	bits
fsk_preamble_detector	enum	GFSK_PREAMBLE_DETECTOR_OFF, _MIN_8BITS, _MIN_16BITS, _MIN_24BITS, _MIN_32BITS	—
fsk_syncword_len	integer	1 - 64	bits
fsk_sync_word	integer (64-bit)	1 - 18 446 744 073 709 551 615	HEX
fsk_address_filtering	enum	GFSK_ADDRESS_FILTERING_DISABLE, _NODE_ADDRESS, _NODE_AND_BROADCAST_ADDRESSES	—
fsk_header_type	enum	GFSK_PKT_FIX_LEN, GFSK_PKT_VAR_LEN, GFSK_PKT_VAR_LEN_SX128X_COMP	—
fsk_crc_type	enum	GFSK_CRC_OFF, _1_BYTE, _2_BYTES, _1_BYTE_INV, _2_BYTES_INV, _3_BYTES, _3_BYTES_INV, _4_BYTES, _4_BYTES_INV	—
fsk_dc_free	enum	GFSK_DC_FREE_OFF, GFSK_DC_FREE_WHITENING, GFSK_DC_FREE_WHITENING_SX128X_COMP	—
fsk_whitening_seed	integer	0 - 4096	HEX
fsk_crc_seed	integer	0 - 4 294 967 295	HEX
fsk_crc_polynomial	integer	0 - 4 294 967 295	HEX
fsk_node_address	integer	0 - 255	HEX
fsk_broadcast_address	integer	0 - 255	HEX

A.5 FLRC parameters (flrc_parameters)

FLRC is available on **LR20xx** shields. It is selectable as a modulation (`packet_type = PKT_TYPE_FLRC`) on **LR2021 only**; on LR2012 / LR2022 the FLRC fields exist but the coding rate is limited to `FLRC_CR_NONE`.

Key	Type	Range / possible values	Fmt
flrc_preamble_len	enum	FLRC_PREAMBLE_LEN_04_BITS, _08, _12, _16, _20, _24, _28, _32_BITS	—
flrc_syncword_len	enum	FLRC_SYNCWORD_LENGTH_OFF, _2_BYTES, _4_BYTES	—
flrc_syncword_match	enum	FLRC_RX_MATCH_SYNCWORD_OFF, _1, _2, _1_OR_2, _3, _1_OR_3, _2_OR_3, _1_OR_2_OR_3	—
flrc_tx_syncword	enum	FLRC_TX_SYNCWORD_NONE, _1, _2, _3	—

flrc_crc_type	enum	FLRC_CRC_OFF, _2_BYTES, _3_BYTES, _4_BYTES	—
flrc_pck_len_mode	enum	FLRC_PKT_VAR_LEN, FLRC_PKT_FIX_LEN	—
flrc_bw	enum	FLRC_BR_2_600_BW_2_666, FLRC_BR_2_080_BW_2_222, FLRC_BR_1_300_BW_1_333, FLRC_BR_1_040_BW_1_333, FLRC_BR_0_650_BW_0_740, FLRC_BR_0_520_BW_0_571, FLRC_BR_0_325_BW_0_357, FLRC_BR_0_260_BW_0_307	—
flrc_cr	enum	FLRC_CR_NONE, FLRC_CR_1_2, FLRC_CR_2_3, FLRC_CR_3_4	—
flrc_pulse_shape	enum	FLRC_PULSE_SHAPE_OFF, FLRC_PULSE_SHAPE_BT_05, FLRC_PULSE_SHAPE_BT_1	—
flrc_syncword_1	integer	0 - 4 294 967 295	HEX
flrc_syncword_2	integer	0 - 4 294 967 295	HEX
flrc_syncword_3	integer	0 - 4 294 967 295	HEX

A.6 OOK parameters (ook_parameters)

Key	Type	Range / possible values	Unit / Fmt
ook_bitrate	integer	0 - 4 294 967 295	bps
ook_bandwidth	integer	0 - 4 294 967 295	Hz
ook_preamble_length	integer	0 - 65 535	bits
ook_pulse_shape_filter	enum	OOK_PULSE_SHAPE_NO_FILTER, OOK_PULSE_SHAPE_FILTER_GAUSSIAN	—
ook_shape_filter_bt	enum	OOK_PULSE_SHAPE_BT_0_3, _0_5, _0_7, _1_0	—
ook_mag_depth	enum	OOK_MAG_DEPTH_FULL, OOK_MAG_DEPTH_UP_TO_20DB	—
ook_address_filtering	enum	OOK_ADDRESS_FILTERING_DISABLE, _NODE_ADDRESS, _NODE_AND_BROADCAST_ADDRESSES	—
ook_header_mode	enum	OOK_HEADER_IMPLICIT, OOK_HEADER_EXPLICIT, OOK_HEADER_16BITS	—
ook_crc_type	enum	OOK_CRC_OFF, _1_BYTE, _2_BYTES, _3_BYTES, _4_BYTES, _1_BYTE_INVERTED, _2_BYTES_INVERTED, _3_BYTES_INVERTED, _4_BYTES_INVERTED	—
ook_encoding	enum	OOK_ENCODING_OFF, OOK_ENCODING_MANCHESTER, OOK_ENCODING_MANCHESTER_INV	—
ook_crc_polynomial	integer	0 - 4 294 967 295	HEX
ook_crc_seed	integer	0 - 4 294 967 295	HEX
ook_sync_word	integer	0 - 4 294 967 295	HEX
ook_sync_word_len_bits	integer	0 - 255	bits
ook_sync_word_bit_order	enum	SHIELD_OOK_SYNCWORD_LSBF, SHIELD_OOK_SYNCWORD_MSBF	—
ook_sync_word_is_encoded	boolean	true / false	—
ook_rx_detector_sfd	enum	SHIELD_OOK_RX_DETECTOR_SFD_TYPE_FALLING_EDGE, ..._RISING_EDGE	—
ook_rx_detector_pattern	integer	0 - 65 535	BIN
ook_rx_detector_pattern_length	integer	0 - 255	bits
ook_rx_detector_pattern_repeat_nb	integer	0 - 255	—
ook_rx_detector_sfd_length	integer	0 - 255	—
ook_node_address	integer	0 - 255	HEX
ook_broadcast_address	integer	0 - 255	HEX
ook_whitening_bit_index	integer	0 - 255	DEC
ook_whitening_polynomial	integer	0 - 65 535	HEX
ook_whitening_seed	integer	0 - 65 535	HEX

A.7 Radio-family differences at a glance

Aspect	LR11xx (LR1110 / LR1120 / LR1121)	LR20xx (LR2021 / LR2022 / LR2012)
Modulations (packet_type)	LoRa, GFSK, OOK	LoRa, GFSK, OOK; FLRC on LR2021 only
Max payload_length	255 bytes	255 bytes (LR2012) / 511 bytes (LR2021,

		LR2022)
LoRa coding rate	Standard + LI rates	Standard + LI + CONVOLUTIONAL rates
Ramp-up time field	lr11xx_ramp_up_time (16-304 µs)	lr20xx_ramp_up_time (2-304 µs)
PA configuration	Driven by tx_output_pwr_in_half_dbm; lr11xx_pa_config for raw control	lr20xx_pa_config (detailed PA control)
FLRC parameters	Not applicable	Present (fully usable on LR2021)